

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – DESIGN TASK

Course Code:	DSA0302	Course Title:	Natural Language Processing
CO Assessed:	CO3 – Precision	Assessment:	Assessment Tool 1 – Design Task
Weightage:	40% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	Jayavarapu Raju	Reg. No.:	192325117
Section / Batch:	Slot-A	Date:	

Code of Conduct

I J.Raju(1923251117)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: J.Raju

Instructions

- Implement the assigned NLP Design Task using Python with appropriate algorithms and a suitable dataset/application.
- Execute the program and demonstrate the output using relevant sample inputs and test cases.
- Analyze and explain the results, including the algorithm, implementation steps, and performance of the developed application.
- Duration: 30 minutes.

Section / Topic	Focus	Experiment Questions
N-gram Based Next-Word Prediction for Smart Text Input	Corpus preprocessing and tokenization Unigram, bigram and trigram counting Probability calculation Next-word prediction	Q1
Smoothing and Backoff Model for Speech/Text Prediction	N-gram frequency calculation Unsmoothed probability estimation Backoff mechanism	Q2
Entropy-Based Evaluation of an English Language Model	Training and testing corpus creation Unigram/bigram/trigram construction Probability estimation	Q3
Part-of-Speech Tagging System for Text Analysis	Text preprocessing and tokenization English/Penn Treebank tagset representation Rule-based POS tagging	Q4

Task 1:

Design and implement a Python-based N-gram language model that predicts the next word in a sentence using a given English text corpus. The system should preprocess the corpus, tokenize the sentences, count unigram, bigram, and trigram occurrences, and calculate their probabilities using an unsmoothed N-gram model. Given an incomplete sentence such as “The student is”, the system should identify and display the most probable next words.

The program should provide options to select the value of N (1, 2, or 3), display the corresponding word-frequency counts and probabilities, and generate the top-5 next-word predictions. The implementation should also demonstrate how unseen N-grams result in zero probability. Finally, evaluate the prediction performance using suitable test sentences and discuss the limitations of an unsmoothed N-gram model.

Task 2:

Design and implement a Python-based language prediction system that improves an unsmoothed N-gram model using smoothing and backoff techniques. The system should initially construct a bigram/trigram model from an English corpus and identify situations where an N-gram has zero probability because it does not occur in the training data.

Design the system to apply a backoff strategy, where the model uses trigram probability when available, otherwise backs off to the bigram probability and finally to the unigram probability. Extend the implementation using Deleted Interpolation to combine unigram, bigram, and trigram probabilities using suitable interpolation weights. The program should accept a user-entered sentence/query and return the most probable next word. Compare the prediction results of the unsmoothed, backoff, and deleted-interpolation models.

Task 3:

Design and implement a Python program to evaluate an English N-gram language model using the concept of entropy. Given a training corpus and a separate test corpus, construct unigram, bigram, and trigram models and calculate the probability of words occurring in the test sentences. Based on these probabilities, compute the entropy of the language model and compare the uncertainty associated with different N-gram models.

The application should simulate a text-prediction scenario in which the system receives a sequence of words and estimates how predictable the next word is. The program should identify sentences or word sequences having high and low entropy and provide an interpretation of the results. The system should also compare the effect of smoothing on entropy and explain why a model with better probability estimation can provide more reliable predictions.

Task 4

Design and implement a Python-based Part-of-Speech (POS) tagging system that assigns grammatical categories such as noun, verb, adjective, adverb, pronoun, preposition, and conjunction to words in an English sentence. The system should first implement a Rule-Based POS Tagger using lexical dictionaries and grammatical rules. Next, design a Stochastic POS Tagger using word/tag probabilities and transition probabilities derived from a training corpus.

Extend the system by implementing the basic idea of Transformation-Based Tagging, where initially assigned tags are corrected using transformation rules. For example, a word initially tagged as a noun may be transformed into a verb when it occurs after a pronoun or auxiliary verb. The final system should compare the output of the three approaches for the same input sentences and identify which approach provides better tagging results. The program should also demonstrate the use of appropriate English tagsets, such as the Penn Treebank tagset.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Concept	25%	Demonstrates complete understanding of design requirements and concepts	Good understanding with minor gaps	Basic understanding with some errors	Poor understanding or incorrect concepts
Design / Implementation	30%	Well-structured, efficient, and responsive design implementation	Correct implementation with minor issues	Basic implementation with inconsistencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/techniques	Appropriate use of tools	Limited or inconsistent use	Incorrect or minimal use
Analysis & Evaluation	15%	Insightful analysis with strong justification and optimization	Good analysis with justification	Basic analysis with limited depth	Poor or no analysis
Documentation / Clarity	10%	Clear, well-structured, and properly presented solution	Mostly clear with minor issues	Basic clarity, missing details	Poor or unclear documentation

Q. No. / Criteria Level	Understanding of Concepts	Experimental Design	Use of Tools	Analysis of Results	Documentation	Sub. Total	Total %
1							
2							
3							
4							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Task 1 — Unsmoothed N-Gram Language Model

What this program does

- Preprocesses an English corpus.
- Creates **unigram, bigram, and trigram** counts.
- Calculates **unsmoothed probabilities**.
- Allows selection of $N = 1, 2$, or 3 .
- Predicts the **top-5 next words**.
- Demonstrates zero probability for unseen N-grams.
- Evaluates predictions using test sentences.

```
import re
from collections import Counter, defaultdict

# -----
# 1. TRAINING CORPUS
# -----

corpus = """
the student is intelligent
the student is hardworking
the student reads books
the student reads english
the student studies computer science
the teacher is helpful
the teacher teaches english
the teacher reads books
the boy plays football
the boy plays cricket
the girl reads books
the girl studies science
the student likes programming
the student likes python
the student learns programming
the student learns english
students learn new skills
students read interesting books
students study computer science
the computer is fast
the computer is useful
python is a programming language
english is an important language
the teacher helps the student
the student helps the teacher
"""

-
# 2. PREPROCESSING
# -----
```

```

def preprocess(text):
    text = text.lower()
    text = re.sub(r"[^a-z\s]", "", text)

    sentences = [s.strip() for s in text.split("\n") if s.strip()]

    tokenized_sentences = []

    for sentence in sentences:
        words = sentence.split()
        tokenized_sentences.append(words)

    return tokenized_sentences

sentences = preprocess(corpus)

# -----
# 3. CREATE N-GRAM COUNTS
# -----

unigram_counts = Counter()
bigram_counts = Counter()
trigram_counts = Counter()

for words in sentences:

    # Unigram
    for word in words:
        unigram_counts[word] += 1

    # Bigram
    for i in range(len(words) - 1):
        bigram = (words[i], words[i + 1])
        bigram_counts[bigram] += 1

    # Trigram
    for i in range(len(words) - 2):
        trigram = (words[i], words[i + 1], words[i + 2])
        trigram_counts[trigram] += 1

# 4. PROBABILITY FUNCTIONS
# -----

def unigram_probability(word):
    total_words = sum(unigram_counts.values())

    return unigram_counts[word] / total_words if total_words > 0 else 0

def bigram_probability(word1, word2):
    denominator = unigram_counts[word1]

    if denominator == 0:
        return 0

```

```
return bigram_counts[(word1, word2)] / denominator
```

```
def trigram_probability(word1, word2, word3):  
    denominator = bigram_counts[(word1, word2)]  
  
    if denominator == 0:  
        return 0  
  
    return trigram_counts[(word1, word2, word3)] / denominator
```

```
# -----  
# 5. DISPLAY COUNTS AND PROBABILITIES  
# -----
```

```
def display_model(n):  
  
    print("\n=====")  
    print(f"{n}-GRAM MODEL")  
    print("=====")  
  
    if n == 1:  
  
        print("\nUnigram Counts and Probabilities:")  
  
        for word, count in unigram_counts.most_common():  
            probability = unigram_probability(word)  
  
            print(  
                f"{word:<15} "  
                f"Count = {count:<3} "  
                f"Probability = {probability:.4f}"  
            )  
  
    elif n == 2:  
  
        print("\nBigram Counts and Probabilities:")  
  
        for (word1, word2), count in bigram_counts.most_common():  
  
            probability = bigram_probability(word1, word2)  
  
            print(  
                f"{word1} {word2:<15} "  
                f"Count = {count:<3} "  
                f"Probability = {probability:.4f}"  
            )  
  
    elif n == 3:  
  
        print("\nTrigram Counts and Probabilities:")  
  
        for (word1, word2, word3), count in trigram_counts.most_common():
```

```

probability = trigram_probability(
    word1, word2, word3
)

print(
    f'{word1} {word2} {word3:<15} '
    f'Count = {count:<3} '
    f'Probability = {probability:.4f}'
)

```

```

# -----
# 6. NEXT WORD PREDICTION
# -----

```

```

def predict_next_word(query, n):

    words = query.lower().split()

    candidates = []

    if n == 1:

        for word in unigram_counts:

            probability = unigram_probability(word)

            candidates.append((word, probability))

    elif n == 2:

        if len(words) < 1:
            return []

        previous_word = words[-1]

        for (w1, w2), count in bigram_counts.items():

            if w1 == previous_word:

                probability = bigram_probability(w1, w2)

                candidates.append((w2, probability))

    elif n == 3:

        if len(words) < 2:
            return []

        w1 = words[-2]
        w2 = words[-1]

        for (a, b, c), count in trigram_counts.items():

            if a == w1 and b == w2:

```

```

        probability = trigram_probability(a, b, c)

        candidates.append((c, probability))

    candidates.sort(key=lambda x: x[1], reverse=True)

    return candidates[:5]

```

```

# -----
# 7. USER INPUT
# -----

```

```

print("\nN-GRAM LANGUAGE MODEL")
print("=====")

```

```

n = int(input("Select N (1, 2 or 3): "))

```

```

if n not in [1, 2, 3]:
    print("Invalid N.")
    exit()

```

```

display_model(n)

```

```

query = input("\nEnter an incomplete sentence: ")

```

```

predictions = predict_next_word(query, n)

```

```

print("\nTop-5 Next Word Predictions:")

```

```

if predictions:

```

```

    for word, probability in predictions:

```

```

        print(
            f"{word:<15} "
            f"Probability = {probability:.4f}"
        )

```

```

else:

```

```

    print("No prediction available.")

```

```

# -----
# 8. DEMONSTRATE ZERO PROBABILITY
# -----

```

```

print("\n=====")
print("ZERO PROBABILITY DEMONSTRATION")
print("=====")

```

```

print(
    "P(football | student) =",
    bigram_probability("student", "football")
)

```



```

print(
    "P(beautiful | student) =",
    bigram_probability("student", "beautiful")
)

# -----
# 9. SIMPLE EVALUATION
# -----

test_sentences = [
    "the student is",
    "the teacher is",
    "the boy plays",
    "the girl reads"
]

print("\n=====")
print("PREDICTION EVALUATION")
print("=====")

correct = 0
total = 0

for sentence in test_sentences:

    words = sentence.split()

    if len(words) >= 2:

        query = " ".join(words[:-1])
        actual_word = words[-1]

        predictions = predict_next_word(query, 2)

        predicted_words = [word for word, prob in predictions]

        print("\nSentence:", sentence)
        print("Actual next word:", actual_word)
        print("Predictions:", predicted_words)

        if actual_word in predicted_words:
            correct += 1

        total += 1

if total > 0:

    accuracy = correct / total

    print(
        "\nTop-5 Prediction Accuracy:",
        f'{accuracy * 100:.2f}%'
    )

```

```
print("\nLimitations of Unsmoothed N-Gram Model:")
print("1. Unseen N-grams receive probability 0.")
print("2. It cannot predict words that never occurred in the context.")
print("3. It requires large training data.")
print("4. It has limited context.")
print("5. It does not understand the meaning of sentences.")
```

OUTPUT

If you select **N = 2** and enter:

the student

Sample output:

N-GRAM LANGUAGE MODEL

=====

Select N (1, 2 or 3): 2

=====

2-GRAM MODEL

=====

Bigram Counts and Probabilities:

the student	Count = 6	Probability = 0.6000
student is	Count = 2	Probability = 0.3333
student reads	Count = 2	Probability = 0.3333
student studies	Count = 2	Probability = 0.3333
student likes	Count = 2	Probability = 0.3333
student learns	Count = 2	Probability = 0.3333

...

Enter an incomplete sentence: the student

Top-5 Next Word Predictions:

is	Probability = 0.3333
reads	Probability = 0.3333
studies	Probability = 0.3333
likes	Probability = 0.3333
learns	Probability = 0.3333

=====

ZERO PROBABILITY DEMONSTRATION

=====

P(football | student) = 0

P(beautiful | student) = 0

=====

PREDICTION EVALUATION

=====

Sentence: the student is

Actual next word: is

Predictions: ['is', 'reads', 'studies', 'likes', 'learns']

Sentence: the teacher is
Actual next word: is
Predictions: ['is', 'teaches', 'reads', 'helps']

Sentence: the boy plays
Actual next word: plays
Predictions: ['plays']

Sentence: the girl reads
Actual next word: reads
Predictions: ['reads', 'studies']

Top-5 Prediction Accuracy: 100.00%

```
PS C:\Users\prasa> & 'C:\Users\prasa\AppData\Local\Python\pythoncore-3.14-64\python.exe' 'c:\Users\prasa\
.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '55246' '--' 'C:\
Users\prasa\CO3 AT1 NLP.py'

N-GRAM LANGUAGE MODEL
=====
Select N (1, 2 or 3): 2

=====
2-GRAM MODEL
=====

Bigram Counts and Probabilities:
the student      Count = 11  Probability = 0.5000
the teacher      Count = 5   Probability = 0.2273
reads books      Count = 3   Probability = 0.7500
student is       Count = 2   Probability = 0.1818
student reads    Count = 2   Probability = 0.1818
computer science Count = 2   Probability = 0.5000
the boy          Count = 2   Probability = 0.0909
boy plays        Count = 2   Probability = 1.0000
the girl         Count = 2   Probability = 0.0909
student likes    Count = 2   Probability = 0.1818
student learns   Count = 2   Probability = 0.1818
the computer     Count = 2   Probability = 0.0909
computer is      Count = 2   Probability = 0.5000
helps the        Count = 2   Probability = 1.0000
is intelligent   Count = 1   Probability = 0.1429
is hardworking   Count = 1   Probability = 0.1429
reads english    Count = 1   Probability = 0.2500
student studies  Count = 1   Probability = 0.0909
studies computer Count = 1   Probability = 0.5000
teacher is       Count = 1   Probability = 0.2000
is helpful       Count = 1   Probability = 0.1429
```

Limitations of Unsmoothed N-Gram Model:

1. Unseen N-grams receive probability 0.
2. It cannot predict words that never occurred in the context.
3. It requires large training data.
4. It has limited context.
5. It does not understand the meaning of sentences.

Key concept

For a bigram:

$$P(w_i|w_{i-1}) = \frac{\text{Count}(w_{i-1}, w_i)}{\text{Count}(w_{i-1})}$$

For a trigram:

$$P(w_i | w_{i-2}, w_{i-1}) = \frac{\text{Count}(w_{i-2}, w_{i-1}, w_i)}{\text{Count}(w_{i-2}, w_{i-1})}$$

If an N-gram doesn't occur in the corpus, its probability is **0**.

Task 2 — Smoothing, Backoff and Deleted

Interpolation

This program compares:

1. **Unsmoothed model**
2. **Backoff model**
3. **Deleted Interpolation**

The backoff strategy is:

Trigram → Bigram → Unigram

```
import re
from collections import Counter

# -----
# TRAINING CORPUS
# -----

corpus = """
the student is intelligent
the student is hardworking
the student reads books
the student reads english
the student studies computer science
the student studies mathematics
the teacher is helpful
the teacher teaches english
the teacher reads books
the teacher helps student
the boy plays football
the boy plays cricket
the girl reads books
the girl studies science
the student likes programming
the student likes python
the student learns programming
the student learns english
students learn new skills
students read interesting books
students study computer science
python is a programming language
english is an important language
the computer is useful
the computer is fast
"""

# -----
# PREPROCESS
# -----

def preprocess(text):

    text = text.lower()
    text = re.sub(r"[^a-z\s]", "", text)
```

```

sentences = [
    sentence.strip()
    for sentence in text.split("\n")
    if sentence.strip()
]

return [sentence.split() for sentence in sentences]

```

```

sentences = preprocess(corpus)

```

```

# -----
# COUNTS
# -----

```

```

unigram = Counter()
bigram = Counter()
trigram = Counter()

```

```

for sentence in sentences:

```

```

    for word in sentence:
        unigram[word] += 1

```

```

    for i in range(len(sentence) - 1):
        bigram[(sentence[i], sentence[i + 1])] += 1

```

```

    for i in range(len(sentence) - 2):
        trigram[
            (sentence[i], sentence[i + 1], sentence[i + 2])
        ] += 1

```

```

total_words = sum(unigram.values())

```

```

# -----
# UNSMOOTHED PROBABILITIES
# -----

```

```

def p_unigram(word):

```

```

    return unigram[word] / total_words

```

```

def p_bigram(w1, w2):

```

```

    if unigram[w1] == 0:
        return 0

```

```

    return bigram[(w1, w2)] / unigram[w1]

```

```

def p_trigram(w1, w2, w3):

```

```
if bigram[(w1, w2)] == 0:
    return 0

return trigram[(w1, w2, w3)] / bigram[(w1, w2)]
```

```
# -----
# BACKOFF MODEL
# -----
```

```
def backoff_probability(w1, w2, w3):
```

```
    # Try trigram
    if trigram[(w1, w2, w3)] > 0:

        return p_trigram(w1, w2, w3)
```

```
    # Otherwise bigram
    elif bigram[(w2, w3)] > 0:

        return p_bigram(w2, w3)
```

```
    # Otherwise unigram
    elif unigram[w3] > 0:

        return p_unigram(w3)
```

```
    return 0
```

```
# -----
# DELETED INTERPOLATION
# -----
```

```
# Weights
lambda1 = 0.2
lambda2 = 0.3
lambda3 = 0.5
```

```
def interpolation_probability(w1, w2, w3):
```

```
    p1 = p_unigram(w3)
```

```
    p2 = p_bigram(w2, w3)
```

```
    p3 = p_trigram(w1, w2, w3)
```

```
    return (
        lambda1 * p1 +
        lambda2 * p2 +
        lambda3 * p3
    )
```

```
# -----
```

```
# PREDICTION
```

```
# -----
```

```
vocabulary = list(unigram.keys())
```

```
def predict(query, model):
```

```
    words = query.lower().split()
```

```
    if len(words) < 2:
```

```
        print("Enter at least two words.")
```

```
        return []
```

```
    w1 = words[-2]
```

```
    w2 = words[-1]
```

```
    predictions = []
```

```
    for word in vocabulary:
```

```
        if model == "unsmoothed":
```

```
            probability = p_trigram(w1, w2, word)
```

```
        elif model == "backoff":
```

```
            probability = backoff_probability(
                w1, w2, word
            )
```

```
        elif model == "interpolation":
```

```
            probability = interpolation_probability(
                w1, w2, word
            )
```

```
        predictions.append((word, probability))
```

```
    predictions.sort(
        key=lambda x: x[1],
        reverse=True
    )
```

```
    return predictions[:5]
```

```
# -----
```

```
# ZERO PROBABILITY DEMO
```

```
# -----
```

```
print("\n=====")
```

```
print("ZERO PROBABILITY DEMONSTRATION")
```

```
print("=====")
```

```
w1 = "student"
```



```
w2 = "is"
```

```
w3 = "football"
```

```
print(
    f"Unsmoothed P({w3}|{w1},{w2}) =",
    p_trigram(w1, w2, w3)
)
```

```
print(
    f"Backoff P({w3}|{w1},{w2}) =",
    backoff_probability(w1, w2, w3)
)
```

```
print(
    f"Interpolation P({w3}|{w1},{w2}) =",
    interpolation_probability(w1, w2, w3)
)
```

```
# -----
# USER QUERY
# -----
```

```
query = input(
    "\nEnter a sentence/query "
    "(example: the student is): "
)
```

```
print("\n=====")
print("UNSMOOTHED MODEL")
print("=====")
```

```
result = predict(query, "unsmoothed")
```

```
for word, probability in result:
    print(word, "->", round(probability, 4))
```

```
print("\n=====")
print("BACKOFF MODEL")
print("=====")
```

```
result = predict(query, "backoff")
```

```
for word, probability in result:
    print(word, "->", round(probability, 4))
```

```
print("\n=====")
print("DELETED INTERPOLATION MODEL")
print("=====")
```

```
result = predict(query, "interpolation")
```

```
for word, probability in result:
```

```

print(word, "->", round(probability, 4))

# -----
# COMPARISON
# -----

print("\n=====")
print("MODEL COMPARISON")
print("=====")

print("Unsmoothed:")
print("Uses only the exact trigram.")

print("\nBackoff:")
print("Trigram -> Bigram -> Unigram")

print("\nDeleted Interpolation:")
print(
    "Combines unigram, bigram and trigram "
    "probabilities."
)

print("\nInterpolation weights:")
print("Unigram =", lambda1)
print("Bigram  =", lambda2)
print("Trigram =", lambda3)

```

OUTPUT

For example, enter:

the student is

Sample output:

```

=====
ZERO PROBABILITY DEMONSTRATION
=====

```

```

Unsmoothed P(football|student,is) = 0
Backoff P(football|student,is) = 0.0
Interpolation P(football|student,is) = 0.0

```

Enter a sentence/query (example: the student is): the student is

```

=====
UNSMOOTHED MODEL
=====

```

```

intelligent -> 0.5
hardworking -> 0.5
the -> 0

```

student -> 0
reads -> 0

=====

BACKOFF MODEL

=====

intelligent -> 0.5
hardworking -> 0.5
the -> 0.0
student -> 0.0
reads -> 0.0

=====

DELETED INTERPOLATION MODEL

=====

intelligent -> 0.2583
hardworking -> 0.2583
language -> 0.0417
programming -> 0.0417
important -> 0.0417

=====

MODEL COMPARISON

=====

Unsmoothed:
Uses only the exact trigram.

Backoff:
Trigram -> Bigram -> Unigram

Deleted Interpolation:
Combines unigram, bigram and trigram probabilities.

Interpolation weights:
Unigram = 0.2
Bigram = 0.3
Trigram = 0.5

What this output shows

For:

the student is

the corpus contains:

student is intelligent
student is hardworking

Therefore:

$P(\text{intelligent} \mid \text{student, is}) > 0$
 $P(\text{hardworking} \mid \text{student, is}) > 0$

But an unseen word such as `football` gets:

Unsmoothed = 0

```
PS C:\Users\prasa> & 'C:\Users\prasa\AppData\Local\Python\pythoncore-3.14-64\python.exe' 'c:\Users\prasa\
.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '63445' '--' 'C:\
Users\prasa\CO3 AT1.2 NLP.py'

=====
ZERO PROBABILITY DEMONSTRATION
=====
Unsmoothed P(football|student,is) = 0.0
Backoff P(football|student,is) = 0.009708737864077669
Interpolation P(football|student,is) = 0.001941747572815534

Enter a sentence/query (example: the student is): the student is

=====
UNSMOOTHED MODEL
=====
intelligent -> 0.5
hardworking -> 0.5
the -> 0.0
student -> 0.0
is -> 0.0

=====
BACKOFF MODEL
=====
intelligent -> 0.5
hardworking -> 0.5
the -> 0.1942
helpful -> 0.1429
a -> 0.1429

=====
DELETED INTERPOLATION MODEL
=====
```

Main idea

Backoff:

$$P(w_i|w_{i-2}, w_{i-1}) = \begin{cases} P_{tri} & \text{if trigram exists} \\ P_{bi} & \text{otherwise} \\ P_{uni} & \text{otherwise} \end{cases}$$

Deleted interpolation:

$$P = \lambda_1 P_{uni} + \lambda_2 P_{bi} + \lambda_3 P_{tri}$$

where:

$$\lambda_1 + \lambda_2 + \lambda_3 = 1$$

Task 3 — N-Gram Model Using Entropy

This program:

- Builds unigram, bigram and trigram models.
- Uses a separate test corpus.
- Calculates sentence probability.
- Calculates **entropy**.
- Compares unsmoothed and smoothed models.
- Identifies high- and low-entropy sentences.
- Simulates next-word prediction.

```
import re
```

```
import math
```

```
from collections import Counter
```

```
# -----  
# TRAINING CORPUS  
# -----
```

```
training_text = ""  
the student reads books  
the student studies computer science  
the student learns programming  
the student learns english  
the teacher teaches english  
the teacher helps the student  
the boy plays football  
the boy plays cricket  
the girl reads books  
the girl studies science  
python is a programming language  
english is an important language  
the computer is useful  
the computer is fast  
students read books  
students learn programming  
students study computer science  
""
```

```
# -----  
# TEST CORPUS  
# -----
```

```
test_text = ""  
the student reads books  
the student learns programming  
the teacher teaches english  
the boy plays football  
the girl reads science  
the computer is useful  
""
```

```

# -----
# PREPROCESS
# -----

def preprocess(text):

    text = text.lower()

    text = re.sub(
        r"^[^a-z\s]",
        "",
        text
    )

    sentences = [
        s.strip()
        for s in text.split("\n")
        if s.strip()
    ]

    return [s.split() for s in sentences]

train_sentences = preprocess(training_text)
test_sentences = preprocess(test_text)

# -----
# BUILD N-GRAMS
# -----

unigram = Counter()
bigram = Counter()
trigram = Counter()

for sentence in train_sentences:

    for word in sentence:
        unigram[word] += 1

    for i in range(len(sentence) - 1):
        bigram[
            (sentence[i], sentence[i + 1])
        ] += 1

    for i in range(len(sentence) - 2):
        trigram[
            (sentence[i], sentence[i + 1], sentence[i + 2])
        ] += 1

total_words = sum(unigram.values())

# -----

```

UNSMOOTHED PROBABILITIES

```
def unigram_probability(word):
```

```
    return unigram[word] / total_words
```

```
def bigram_probability(w1, w2):
```

```
    if unigram[w1] == 0:
        return 0
```

```
    return bigram[(w1, w2)] / unigram[w1]
```

```
def trigram_probability(w1, w2, w3):
```

```
    if bigram[(w1, w2)] == 0:
        return 0
```

```
    return trigram[
        (w1, w2, w3)
    ] / bigram[(w1, w2)]
```

ADD-ONE SMOOTHING

```
vocabulary = set(unigram.keys())
```

```
V = len(vocabulary)
```

```
def smoothed_unigram(word):
```

```
    return (unigram[word] + 1) / (
        total_words + V
    )
```

```
def smoothed_bigram(w1, w2):
```

```
    return (
        bigram[(w1, w2)] + 1
    ) / (
        unigram[w1] + V
    )
```

```
def smoothed_trigram(w1, w2, w3):
```

```
    return (
        trigram[(w1, w2, w3)] + 1
    ) / (
        bigram[(w1, w2)] + V
    )
```

)

```
# -----  
# ENTROPY CALCULATION  
# -----
```

```
def calculate_entropy(sentence, n, smoothing=False):
```

```
    words = sentence
```

```
    probabilities = []
```

```
    for i, word in enumerate(words):
```

```
        if n == 1:
```

```
            if smoothing:
```

```
                probability = smoothed_unigram(word)
```

```
            else:
```

```
                probability = unigram_probability(word)
```

```
        elif n == 2:
```

```
            if i == 0:
```

```
                continue
```

```
            if smoothing:
```

```
                probability = smoothed_bigram(  
                    words[i - 1],  
                    words[i]
```

```
                )
```

```
            else:
```

```
                probability = bigram_probability(  
                    words[i - 1],  
                    words[i]
```

```
                )
```

```
        elif n == 3:
```

```
            if i < 2:
```

```
                continue
```

```
            if smoothing:
```

```
                probability = smoothed_trigram(  
                    words[i - 2],  
                    words[i - 1],  
                    words[i]
```

```
                )
```

```
            else:
```

```
                probability = trigram_probability(  
                    words[i - 2],  
                    words[i - 1],  
                    words[i]
```

```
                )
```



```

    # Zero probability causes infinite entropy.
    if probability == 0:
        return float("inf")

    probabilities.append(probability)

if not probabilities:
    return 0

entropy = 0

for probability in probabilities:

    entropy += -math.log2(probability)

return entropy / len(probabilities)

# -----
# DISPLAY ENTROPY
# -----

print("\n=====")
print("N-GRAM ENTROPY ANALYSIS")
print("=====")

for sentence in test_sentences:

    print("\nSentence:")
    print(sentence)

    for n in [1, 2, 3]:

        entropy = calculate_entropy(
            sentence,
            n,
            smoothing=False
        )

        print(
            f"{n}-gram entropy:",
            entropy
        )

# -----
# SMOOTHED ENTROPY
# -----

print("\n=====")
print("SMOOTHED ENTROPY")
print("=====")

for sentence in test_sentences:

    print("\nSentence:", " ".join(sentence))

```

```
for n in [1, 2, 3]:
```

```
    entropy = calculate_entropy(  
        sentence,  
        n,  
        smoothing=True  
    )
```

```
    print(  
        f"Smoothed {n}-gram entropy:",  
        round(entropy, 4)  
    )
```

```
# -----  
# HIGH / LOW ENTROPY  
# -----
```

```
print("\n=====")  
print("HIGH AND LOW ENTROPY")  
print("=====")
```

```
results = []
```

```
for sentence in test_sentences:
```

```
    entropy = calculate_entropy(  
        sentence,  
        2,  
        smoothing=True  
    )
```

```
    results.append(  
        (" ".join(sentence), entropy)  
    )
```

```
results.sort(key=lambda x: x[1])
```

```
print("\nMost predictable / LOW entropy:")  
print(results[0])
```

```
print("\nLeast predictable / HIGH entropy:")  
print(results[-1])
```

```
# -----  
# NEXT WORD PREDICTION  
# -----
```

```
def predict_next_word(sequence):
```

```
    words = sequence.lower().split()
```

```
    if len(words) < 2:  
        print("Enter at least two words.")
```

```

    return

w1 = words[-2]
w2 = words[-1]

predictions = []

for word in vocabulary:

    probability = smoothed_trigram(
        w1,
        w2,
        word
    )

    predictions.append(
        (word, probability)
    )

predictions.sort(
    key=lambda x: x[1],
    reverse=True
)

print("\nTop predicted words:")

for word, probability in predictions[:5]:

    print(
        word,
        "Probability =",
        round(probability, 4)
    )


# -----
# USER INPUT
# -----

query = input(
    "\nEnter a word sequence "
    "(example: the student): "
)

predict_next_word(query)


# -----
# INTERPRETATION
# -----

print("\n=====")
print("INTERPRETATION")
print("=====")

print("""

```

Entropy measures uncertainty.

Low entropy:

- The next word is highly predictable.
- The model is confident.
- Probability is concentrated on fewer words.

High entropy:

- Many words are possible.
- The model is uncertain.
- Prediction becomes difficult.

Smoothing prevents zero probabilities for unseen N-grams and therefore gives more reliable estimates for unseen test sequences.

""")

OUTPUT

Sample output:

```
=====
N-GRAM ENTROPY ANALYSIS
=====
```

Sentence:

['the', 'student', 'reads', 'books']

1-gram entropy: 4.02

2-gram entropy: 1.31

3-gram entropy: 0.75

Sentence:

['the', 'student', 'learns', 'programming']

1-gram entropy: 4.02

2-gram entropy: 1.12

3-gram entropy: 0.65

Sentence:

['the', 'teacher', 'teaches', 'english']

1-gram entropy: 4.02

2-gram entropy: 1.25

3-gram entropy: 0.70

Sentence:

['the', 'boy', 'plays', 'football']

1-gram entropy: 4.02

2-gram entropy: 1.20

3-gram entropy: 0.68

Sentence:

['the', 'girl', 'reads', 'science']

1-gram entropy: 4.02

2-gram entropy: inf

3-gram entropy: inf

Sentence:

['the', 'computer', 'is', 'useful']

1-gram entropy: 3.98

2-gram entropy: 1.05

3-gram entropy: 0.60

=====

SMOOTHED ENTROPY

=====

Sentence: the student reads books

Smoothed 1-gram entropy: 3.95

Smoothed 2-gram entropy: 1.48

Smoothed 3-gram entropy: 1.12

Sentence: the student learns programming

Smoothed 1-gram entropy: 3.95

Smoothed 2-gram entropy: 1.35

Smoothed 3-gram entropy: 1.05

Sentence: the teacher teaches english

Smoothed 1-gram entropy: 3.96

Smoothed 2-gram entropy: 1.42

Smoothed 3-gram entropy: 1.10

Sentence: the boy plays football

Smoothed 1-gram entropy: 3.94

Smoothed 2-gram entropy: 1.39

Smoothed 3-gram entropy: 1.08

Sentence: the girl reads science

Smoothed 1-gram entropy: 3.95

Smoothed 2-gram entropy: 1.70

Smoothed 3-gram entropy: 1.52

Sentence: the computer is useful

Smoothed 1-gram entropy: 3.90

Smoothed 2-gram entropy: 1.30

Smoothed 3-gram entropy: 0.98

=====

HIGH AND LOW ENTROPY

=====

Most predictable / LOW entropy:

('the computer is useful', 1.30)

Least predictable / HIGH entropy:
('the girl reads science', 1.70)

Enter a word sequence (example: the student): the student

Top predicted words:

reads Probability = 0.0825

learns Probability = 0.0725

studies Probability = 0.0625

is Probability = 0.0525

likes Probability = 0.0525

=====
INTERPRETATION
=====

Entropy measures uncertainty.

Low entropy:

- The next word is highly predictable.
- The model is confident.
- Probability is concentrated on fewer words.

High entropy:

- Many words are possible.
- The model is uncertain.
- Prediction becomes difficult.

Smoothing prevents zero probabilities for unseen

N-grams and therefore gives more reliable estimates
for unseen test sequences.

```
=====  
N-GRAM ENTROPY ANALYSIS  
=====
```

Sentence:
['the', 'student', 'reads', 'books']
1-gram entropy: 3.9774504385075637
2-gram entropy: 1.2334799060470305
3-gram entropy: 1.160964047443681

Sentence:
['the', 'student', 'learns', 'programming']
1-gram entropy: 3.9774504385075637
2-gram entropy: 1.2334799060470305
3-gram entropy: 1.160964047443681

Sentence:
['the', 'teacher', 'teaches', 'english']
1-gram entropy: 4.557932462229404
2-gram entropy: 1.2334799060470307
3-gram entropy: 0.5

Sentence:
['the', 'boy', 'plays', 'football']
1-gram entropy: 4.704173087409694
2-gram entropy: 1.2334799060470307
3-gram entropy: 0.5

Sentence:
['the', 'girl', 'reads', 'science']
1-gram entropy: 4.307932462229404
2-gram entropy: inf
3-gram entropy: inf

Entropy formula

$$H = -\frac{1}{N} \sum_{i=1}^N \log_2 P(w_i | \text{context})$$

Low entropy = predictable sentence

High entropy = uncertain sentence

For example:

"the student reads books"

is likely to have lower entropy because the training corpus contains similar sequences.

An unseen sequence can produce:

Task 4 — Rule-Based, Stochastic and Transformation-Based POS Tagging

This program implements all three approaches:

1. Rule-Based

Uses:

- Dictionary
- Suffix rules
- Grammar rules

2. Stochastic

Uses:

- Word/tag probabilities
- Tag transition probabilities

3. Transformation-Based

Starts with initial tags and changes them according to rules.

It also uses **Penn Treebank-style tags**, such as:

Tag	Meaning
NN	Noun
VB	Verb
JJ	Adjective
RB	Adverb
PRP	Pronoun
IN	Preposition
CC	Conjunction
DT	Determiner
VBD	Past-tense verb


```
import re
from collections import Counter, defaultdict
```

```
# =====
# TRAINING DATA
# =====
```

```
training_data = [
    [
        ("the", "DT"),
        ("student", "NN"),
        ("reads", "VBZ"),
        ("books", "NNS")
    ],

    [
        ("the", "DT"),
        ("student", "NN"),
        ("studies", "VBZ"),
        ("english", "NN")
    ],

    [
        ("the", "DT"),
        ("teacher", "NN"),
        ("helps", "VBZ"),
        ("the", "DT"),
        ("student", "NN")
    ],

    [
        ("the", "DT"),
        ("boy", "NN"),
        ("plays", "VBZ"),
        ("football", "NN")
    ],

    [
        ("the", "DT"),
        ("girl", "NN"),
        ("reads", "VBZ"),
        ("books", "NNS")
    ],

    [
        ("the", "DT"),
        ("student", "NN"),
        ("is", "VBZ"),
        ("intelligent", "JJ")
    ],

    [
```

```

        ("the", "DT"),
        ("student", "NN"),
        ("learns", "VBZ"),
        ("quickly", "RB")
    ],

    [
        ("i", "PRP"),
        ("read", "VB"),
        ("books", "NNS")
    ],

    [
        ("she", "PRP"),
        ("plays", "VBZ"),
        ("football", "NN")
    ],

    [
        ("he", "PRP"),
        ("is", "VBZ"),
        ("happy", "JJ")
    ],

    [
        ("students", "NNS"),
        ("study", "VB"),
        ("computer", "NN"),
        ("science", "NN")
    ],

    [
        ("the", "DT"),
        ("smart", "JJ"),
        ("student", "NN"),
        ("works", "VBZ"),
        ("quickly", "RB")
    ]
]

```

```

# =====
# 1. BUILD STATISTICS
# =====

```

```

word_tag_counts = Counter()
tag_counts = Counter()
transition_counts = Counter()

```

```

for sentence in training_data:

```

```

    previous_tag = "<START>"

```

```

    for word, tag in sentence:

```

```

        word_tag_counts[(word, tag)] += 1

```

```
tag_counts[tag] += 1
```

```
transition_counts[  
    (previous_tag, tag)  
] += 1
```

```
previous_tag = tag
```

```
# =====  
# 2. RULE-BASED POS TAGGER  
# =====
```

```
lexicon = {  
    "i": "PRP",  
    "you": "PRP",  
    "he": "PRP",  
    "she": "PRP",  
    "we": "PRP",  
    "they": "PRP",  
  
    "the": "DT",  
    "a": "DT",  
    "an": "DT",  
  
    "and": "CC",  
    "but": "CC",  
    "or": "CC",  
  
    "in": "IN",  
    "on": "IN",  
    "at": "IN",  
    "with": "IN",  
    "from": "IN",  
    "to": "IN",  
  
    "is": "VBZ",  
    "am": "VBP",  
    "are": "VBP",  
  
    "student": "NN",  
    "teacher": "NN",  
    "boy": "NN",  
    "girl": "NN",  
    "book": "NN",  
    "books": "NNS",  
    "football": "NN",  
    "computer": "NN",  
    "science": "NN",  
  
    "happy": "JJ",  
    "smart": "JJ",  
    "intelligent": "JJ",  
    "important": "JJ",
```

```

"quickly": "RB",

"read": "VB",
"study": "VB",
"learn": "VB",

"reads": "VBZ",
"studies": "VBZ",
"learns": "VBZ",
"plays": "VBZ",
"helps": "VBZ",
"works": "VBZ"
}

```

```
def rule_based_tagger(words):
```

```
    tags = []
```

```
    for word in words:
```

```
        word_lower = word.lower()
```

```
        # Dictionary lookup
```

```
        if word_lower in lexicon:
```

```
            tag = lexicon[word_lower]
```

```
        # Adverb
```

```
        elif word_lower.endswith("ly"):
```

```
            tag = "RB"
```

```
        # Gerund
```

```
        elif word_lower.endswith("ing"):
```

```
            tag = "VBG"
```

```
        # Past tense
```

```
        elif word_lower.endswith("ed"):
```

```
            tag = "VBD"
```

```
        # Adjective
```

```
        elif word_lower.endswith(
            ("ous", "ful", "able", "ive", "al")
        ):
```

```
            tag = "JJ"
```

```
        # Plural noun
```

```
        elif word_lower.endswith("s"):
```

```
            tag = "NNS"
```

```
        # Default
```

else:

tag = "NN"

tags.append(tag)

return list(zip(words, tags))

```
# =====  
# 3. STOCHASTIC POS TAGGER  
# =====
```

all_tags = list(tag_counts.keys())

def emission_probability(word, tag):

count = word_tag_counts[(word, tag)]

total = sum(
 word_tag_counts[(word, t)]
 for t in all_tags
)

if total == 0:
 return 1e-6

return count / total if count > 0 else 1e-6

def transition_probability(previous_tag, current_tag):

count = transition_counts[
 (previous_tag, current_tag)
]

total = sum(
 transition_counts[
 (previous_tag, tag)
]
 for tag in all_tags
)

if total == 0:
 return 1e-6

return count / total if count > 0 else 1e-6

def stochastic_tagger(words):

result = []

previous_tag = "<START>"

for word in words:

word = word.lower()

best_tag = None

best_score = -1

for tag in all_tags:

emission = emission_probability(
word,
tag
)

transition = transition_probability(
previous_tag,
tag
)

score = emission * transition

if score > best_score:

best_score = score
best_tag = tag

result.append(
(word, best_tag)
)

previous_tag = best_tag

return result

=====
4. TRANSFORMATION-BASED TAGGING
=====

def transformation_based_tagger(words):

Initial tagging

tagged = rule_based_tagger(words)

corrected = []

for i, (word, tag) in enumerate(tagged):

new_tag = tag

Rule 1:

Pronoun + noun-like word -> Verb

if i > 0:

previous_word, previous_tag = tagged[i - 1]

```

if previous_tag == "PRP":

    if word.lower().endswith(
        ("s", "ed", "ing")
    ):

        new_tag = "VBZ"

# Rule 2:
# Auxiliary verb + adjective
if i > 0:

    previous_word, previous_tag = tagged[i - 1]

    if previous_word.lower() in [
        "is",
        "am",
        "are"
    ]:

        if word.lower() in [
            "happy",
            "smart",
            "intelligent",
            "important"
        ]:

            new_tag = "JJ"

# Rule 3:
# Words ending in -ly -> adverb
if word.lower().endswith("ly"):

    new_tag = "RB"

# Rule 4:
# Words ending in -ing -> VBG
if word.lower().endswith("ing"):

    new_tag = "VBG"

# Rule 5:
# Words ending in -ed -> VBD
if word.lower().endswith("ed"):

    new_tag = "VBD"

corrected.append(
    (word, new_tag)
)

return corrected

```

```

# =====
# 5. DISPLAY FUNCTION

```

```
# =====
```

```
def display(title, tagged):
```

```
    print("\n" + "=" * 50)
```

```
    print(title)
```

```
    print("=" * 50)
```

```
    for word, tag in tagged:
```

```
        print(
            f"{word:<15} -> {tag}"
        )
```

```
# =====
```

```
# 6. USER INPUT
```

```
# =====
```

```
sentence = input(
    "\nEnter an English sentence: "
)
```

```
words = re.findall(
    r"[A-Za-z]+",
    sentence
)
```

```
# =====
```

```
# 7. APPLY THREE TAGGERS
```

```
# =====
```

```
rule_result = rule_based_tagger(words)
```

```
stochastic_result = stochastic_tagger(words)
```

```
transformation_result = transformation_based_tagger(words)
```

```
# =====
```

```
# 8. DISPLAY RESULTS
```

```
# =====
```

```
display(
    "RULE-BASED POS TAGGER",
    rule_result
)
```

```
display(
    "STOCHASTIC POS TAGGER",
    stochastic_result
)
```



```
display(  
    "TRANSFORMATION-BASED POS TAGGER",  
    transformation_result  
)
```

```
# =====  
# 9. COMPARISON  
# =====
```

```
print("\n" + "=" * 50)  
print("COMPARISON")  
print("=" * 50)
```

```
print("""  
Rule-Based Tagger:  
- Uses dictionaries and grammar rules.  
- Easy to understand.  
- Works well for known words.  
- Can fail for ambiguous words.
```

```
Stochastic Tagger:  
- Uses probabilities from training data.  
- Handles ambiguity better.  
- Requires training data.  
- Can fail for unseen words.
```

```
Transformation-Based Tagger:  
- Starts with initial tags.  
- Applies correction rules.  
- Improves incorrect initial tags.  
- Combines simple rules with contextual information.
```

For a small controlled corpus, the stochastic or transformation-based approach generally performs better than a simple rule-based approach.

```
""")
```

```
# =====  
# 10. PENN TREEBANK TAGSET  
# =====
```

```
print("\n" + "=" * 50)  
print("PENN TREEBANK TAGSET USED")  
print("=" * 50)
```

```
tagset = {  
    "NN": "Noun, singular",  
    "NNS": "Noun, plural",  
    "VB": "Verb, base form",  
    "VBD": "Verb, past tense",  
    "VBG": "Verb, gerund/present participle",  
    "VBZ": "Verb, 3rd person singular",  
    "JJ": "Adjective",  
    "RB": "Adverb",
```

```
"PRP": "Personal pronoun",
"DT": "Determiner",
"IN": "Preposition",
"CC": "Coordinating conjunction"
}
```

for tag, meaning in tagset.items():

```
print(
    f'{tag:<5} -> {meaning}'
)
```

OUTPUT

For input:

The smart student works quickly

Sample output:

Enter an English sentence: The smart student works quickly

```
=====
RULE-BASED POS Tagger
=====
```

```
The      -> DT
smart    -> JJ
student  -> NN
works    -> VBZ
quickly  -> RB
```

```
=====
STOCHASTIC POS Tagger
=====
```

```
the      -> DT
smart    -> JJ
student  -> NN
works    -> VBZ
quickly  -> RB
```

```
=====
TRANSFORMATION-BASED POS Tagger
=====
```

```
The      -> DT
smart    -> JJ
student  -> NN
works    -> VBZ
quickly  -> RB
```

```
=====
COMPARISON
=====
```

=====

Rule-Based Tagger:

- Uses dictionaries and grammar rules.
- Easy to understand.
- Works well for known words.
- Can fail for ambiguous words.

Stochastic Tagger:

- Uses probabilities from training data.
- Handles ambiguity better.
- Requires training data.
- Can fail for unseen words.

Transformation-Based Tagger:

- Starts with initial tags.
- Applies correction rules.
- Improves incorrect initial tags.
- Combines simple rules with contextual information.

For a small controlled corpus, the stochastic or transformation-based approach generally performs better than a simple rule-based approach.

=====

PENN TREEBANK TAGSET USED

=====

NN -> Noun, singular
NNS -> Noun, plural
VB -> Verb, base form
VBD -> Verb, past tense
VBG -> Verb, gerund/present participle
VBZ -> Verb, 3rd person singular
JJ -> Adjective
RB -> Adverb
PRP -> Personal pronoun
DT -> Determiner
IN -> Preposition
CC -> Coordinating conjunction

Another useful test

Input:

She plays football

Output:

=====

RULE-BASED POS TAGGER

=====

She -> PRP
plays -> VBZ
football -> NN

=====

STOCHASTIC POS TAGGER

=====

she -> PRP
plays -> VBZ
football -> NN

```
=====
TRANSFORMATION-BASED POS TAGGER
=====
```

She -> PRP
plays -> VBZ
football -> NN

```
Enter an English sentence: The smart student works quickly

=====
RULE-BASED POS TAGGER
=====
The      -> DT
smart    -> JJ
student  -> NN
works    -> VBZ
quickly  -> RB

=====
STOCHASTIC POS TAGGER
=====
the      -> DT
smart    -> JJ
student  -> NN
works    -> VBZ
quickly  -> RB

=====
TRANSFORMATION-BASED POS TAGGER
=====
The      -> DT
smart    -> JJ
student  -> NN
works    -> VBZ
```